

Spring Bean Lifecycle

1. What is Bean Lifecycle?

In normal Java, when we create an object manually, we are responsible for everything.

```
PaymentService paymentService = new PaymentService();
```

That means we handle the complete object journey ourselves:

```
Create object
→ set required values
→ call required methods
→ clean up resources if needed
```

But in Spring, we usually do not create important application objects manually. Instead, we tell Spring which classes or methods should be managed by the container.

Example using `@Component`:

```
@Component
public class PaymentService {
}
```

Example using `@Bean`:

```
@Configuration
public class AppConfig {

    @Bean
    public PaymentService paymentService() {
        return new PaymentService();
    }
}
```

```
}  
}
```

Now this object becomes Spring's responsibility.

Spring will:

```
discover the bean  
→ create the object  
→ inject dependencies  
→ call lifecycle methods  
→ keep the bean ready  
→ destroy it when required
```

2. Simple Definition

Bean lifecycle means the complete journey of a Spring-managed object from the moment Spring discovers its definition until the moment Spring destroys it.

A Spring-managed object is called a **bean**.

So whenever we say "bean lifecycle," we are talking about:

How Spring creates, prepares, manages, and destroys a bean.

3. Complete Bean Lifecycle Flow

```
Spring container starts  
↓  
Reads configuration / annotations  
↓  
Creates BeanDefinition  
↓  
Instantiates bean object  
↓  
Injects dependencies  
↓  
Calls Aware interfaces  
↓      ↓
```

```
Runs initialization callbacks
    ↓      ↓
Bean is ready to use
    ↓
Application uses the bean
    ↓      ↓
Runs destruction callbacks
    ↓
Bean is removed
```

4. Step 0: Spring Container Starts

When we write:

```
ApplicationContext context =
    new AnnotationConfigApplicationContext(AppConfig.class);
```

we are not passing an object of `AppConfig`.

We are passing the **class metadata** of `AppConfig`.

Spring uses reflection to inspect that class.

It checks things like:

```
Does this class have @Configuration?
Does it have @ComponentScan?
Does it have @Bean methods?
Which package should be scanned?
Which bean definitions should be registered?
```

This is where the Spring container starts understanding the application structure.

5. Step 1: Bean Definition is Read

Before creating objects, Spring first collects metadata about beans.

This metadata is called a **BeanDefinition**.

A bean definition contains information like:

```
Bean name: paymentService  
Bean class: PaymentService  
Scope: singleton  
Lazy: false  
Dependencies: check later  
Init method: check later  
Destroy method: check later
```

At this stage, the object is not necessarily created.

Spring is only registering information about the bean.

Why does Spring need BeanDefinition first?

Because Spring is not creating just one object.

It is building a complete object network.

For example:

```
OrderService depends on PaymentService  
PaymentService depends on PaymentGateway  
PaymentGateway depends on ApiClient
```

Spring first needs to know:

```
Which classes should I manage?  
What is the bean name?  
What is the scope?  
What dependencies does it need?  
Is it lazy or eager?  
Does it have init or destroy methods?
```

Only after understanding this structure does Spring start creating objects.

Where does Spring read bean definitions from?

Spring can read bean definitions from different sources.

Common sources are:

1. Annotation-based configuration

```
@Configuration
@ComponentScan
public class AppConfig {
}
```

```
@Component
public class PaymentService {
}
```

1. Java-based `@Bean` configuration

```
@Bean
public PaymentService paymentService() {
    return new PaymentService();
}
```

1. XML-based configuration

```
<bean id="paymentService" class="in.strikes.PaymentService"/>
```

6. Step 2: Bean Object is Instantiated

After reading the bean definition, Spring creates the actual object.

This step is called **instantiation**.

Example:

```
@Component
public class PaymentService {
```

```

    public PaymentService() {
        System.out.println("PaymentService con
d");
    }
}

```

When Spring creates this bean, the constructor is called.

```

Instantiation = object is created
Initialization = object is prepared after creation

```

These two are not the same.

A common beginner mistake is thinking that object creation and initialization are the same thing.

They are different phases in the Spring lifecycle.

7. Step 3: Dependencies are Injected

After or during object creation, Spring supplies the required dependencies.

This is called **dependency injection**.

The exact timing depends on the type of injection.

Constructor Injection

```

@Component
public class OrderService {

    private final PaymentService paymentService;

    public OrderService(PaymentService paymentService) {
        this.paymentService = paymentService;
    }
}

```

```
}  
}
```

In constructor injection, the dependency is supplied while cr

Flow:

```
Spring needs OrderService  
    ↓  
OrderService needs PaymentService  
    ↓  
Spring creates or finds PaymentService  
    ↓  
Spring passes PaymentService into OrderService constructor
```

Setter Injection

```
@Component  
public class OrderService {  
  
    private PaymentService paymentService;  
  
    @Autowired  
    public void setPaymentService(PaymentService paymentService) {  
        this.paymentService = paymentService;  
    }  
}
```

In setter injection:

```
Object is created first  
    ↓  
Spring calls the setter method  
    ↓  
Dependency is assigned
```

Field Injection

```
@Component
public class OrderService {

    @Autowired
    private PaymentService paymentService;
}
```

In field injection:

```
Object is created first
    ↓
Spring uses reflection
    ↓
Dependency is injected directly into the field
```

Field injection is common in examples, but constructor injection is generally preferred for clean and testable code.

8. Step 4: Aware Interfaces are Called

Sometimes a bean wants to know something about the Spring container.

For example:

```
What is my bean name?
Which BeanFactory created me?
Which ApplicationContext am I running inside?
```

Spring provides this information using **Aware interfaces**.

Aware interfaces are callback interfaces.

A callback means:

```
Spring automatically calls a method at the right time.
```

Common Aware Interfaces

Some common Aware interfaces are:

```
BeanNameAware  
ApplicationContextAware
```

Example:

```
@Component("myCustomBeanName")  
public class MyBean implements BeanNameAware, ApplicationContextAware {  
  
    public MyBean() {  
        System.out.println("1. Constructor called");  
    }  
  
    @Override  
    public void setBeanName(String name) {  
        System.out.println("2. BeanNameAware called");  
        System.out.println("Bean name: " + name);  
    }  
  
    @Override  
    public void setApplicationContext(ApplicationContext applicationContext) {  
        System.out.println("3. ApplicationContextAware called");  
        System.out.println("ApplicationContext: " + applicationContext.getClass().getSimpleName());  
    }  
}
```

Output idea:

```
1. Constructor called
2. BeanNameAware called
   Bean name: myCustomBeanName
3. ApplicationContextAware called
   ApplicationContext: AnnotationConfigApplicationContext
```

Why is the method called `setBeanName()` and not `getBeanName()` ?

Because Spring is giving the bean some information.

The direction is:

```
Spring container → bean
```

So Spring calls the method and sets the bean name into that bean.

If we call `setBeanName()` manually, it will behave like a normal Java method.

It will not change the actual name registered inside the Spring container.

Do we use Aware interfaces in normal business logic?

Usually, no.

In normal service classes, we rarely need them.

They are more useful when we are building:

```
framework-level code
logging utilities
custom libraries
infrastructure components
```

For learning Spring lifecycle, they are useful because they show how Spring communicates container information to beans.

9. Step 5: Initialization Callbacks

At this point:

```
Bean object is created  
Dependencies are injected  
Aware callbacks are completed
```

Now Spring gives the bean a chance to run some startup logic before it is used.

This is called the **initialization phase**.

Initialization logic can include:

```
validate configuration  
initialize internal cache  
check required setup  
prepare resources  
fill a HashMap with calculated values  
log startup state
```

Common Ways to Write Initialization Logic

There are three common ways.

Option 1: `@PostConstruct`

```
import jakarta.annotation.PostConstruct;  
  
@Component  
public class PaymentService {  
  
    @PostConstruct  
    public void init() {  
        System.out.println("Bean initialized");  
    }  
}
```

```
}  
}
```

`@PostConstruct` tells Spring:

After dependencies are injected, call this method once.

This is clean and commonly used.

In Spring 6 and Spring 7, use:

```
import jakarta.annotation.PostConstruct;
```

If you are using only Spring Core/Spring Context and not Spring Boot, you may need to add the Jakarta annotation dependency separately.

Maven dependency:

```
<dependency>  
  <groupId>jakarta.annotation</groupId>  
  <artifactId>jakarta.annotation-api</artifactId>  
  <version>3.0.0</version>  
</dependency>
```

Option 2: `InitializingBean`

```
import org.springframework.beans.factory.InitializingBean;  
  
@Component  
public class PaymentService implements InitializingBean {  
  
    @Override  
    public void afterPropertiesSet() {  
        System.out.println("Bean initialized");  
    }  
}
```

```
}  
}
```

Here the bean implements a Spring interface.

Spring calls `afterPropertiesSet()` during the initialization phase

This works, but it tightly couples our class with Spring.

Option 3: Custom `initMethod`

```
@Configuration  
public class AppConfig {  
  
    @Bean(initMethod = "start")  
    public PaymentService paymentService() {  
        return new PaymentService();  
    }  
}
```

```
public class PaymentService {  
  
    public void start() {  
        System.out.println("Bean initialized");  
    }  
}
```

Here we define our own method name and tell Spring to call it after dependency injection.

This keeps the class free from Spring-specific interfaces.

Initialization Callback Order

If all three are used together, the usual order is:

```
@PostConstruct
    ↓
InitializingBean.afterPropertiesSet()
    ↓
custom initMethod
```

In real projects, we usually do not use all three together.

Pick one clear approach.

10. Step 6: Bean is Ready to Use

After initialization is complete, the bean is fully ready.

Now other beans can use it.

For singleton beans, Spring stores the object and reuses the same instance again and again.

```
Bean created
→ dependencies injected
→ initialization completed
→ bean ready
```

At this stage, the bean can safely handle business logic.

11. Step 7: Destruction Callbacks

When the Spring container is shutting down, Spring gives beans a chance to clean up resources.

This is called the **destruction phase**.

Cleanup logic can include:

```
close database connection
stop background thread
release file handle
```

```
clear resource  
flush pending data
```

Common Ways to Write Destruction Logi

There are three common ways.

Option 1: **@PreDestroy**

```
import jakarta.annotation.PreDestroy;  
  
@Component  
public class PaymentService {  
  
    @PreDestroy  
    public void cleanup() {  
        System.out.println("Cleaning up bean");  
    }  
}
```

Spring calls this method before destroying the bean.

Option 2: **DisposableBean**

```
import org.springframework.beans.factory.DisposableBean;  
  
@Component  
public class PaymentService implements DisposableBean {  
  
    @Override  
    public void destroy() {  
        System.out.println("Cleaning up bean");  
    }  
}
```

```
}  
}
```

This works, but it couples our class with Spring.

Option 3: Custom `destroyMethod`

```
@Configuration  
public class AppConfig {  
  
    @Bean(destroyMethod = "stop")  
    public PaymentService paymentService() {  
        return new PaymentService();  
    }  
}
```

```
public class PaymentService {  
  
    public void stop() {  
        System.out.println("Cleaning up bean");  
    }  
}
```

Destruction Callback Order

If all three are used together, the usual order is:

```
@PreDestroy  
    ↓  
DisposableBean.destroy()  
    ↓  
custom destroyMethod
```

Again, in real projects, we usually choose one approach.

12. Full Lifecycle Summary

1. Spring container starts
2. Spring reads configuration and annotations
3. Spring creates BeanDefinition
4. Spring instantiates the bean object
5. Spring injects dependencies
6. Spring calls Aware interfaces
7. Spring runs initialization callbacks
 - @PostConstruct
 - InitializingBean
 - custom initMethod
8. Bean is ready to use
9. Application uses the bean
10. Spring runs destruction callbacks
 - @PreDestroy
 - DisposableBean
 - custom destroyMethod
11. Bean is removed

13. Singleton Bean Lifecycle

By default, Spring beans are singleton scoped.

Singleton means:

Spring creates only one object for that bean.

For singleton beans, Spring manages the complete lifecycle.

creation
→ dependency injection

- initialization
- usage
- destruction

For example:

```
@Component
public class PaymentService {

    public PaymentService() {
        System.out.println("Constructor called");
    }

    @PostConstruct
    public void init() {
        System.out.println("@PostConstruct called");
    }

    @PreDestroy
    public void destroy() {
        System.out.println("@PreDestroy called");
    }
}
```

When the context starts:

```
Constructor called
@PostConstruct called
```

When the context closes:

```
@PreDestroy called
```

So for singleton beans:

```
Spring creates it.  
Spring stores it.  
Spring reuses it.  
Spring destroys it.
```

14. Prototype Bean Lifecycle

Prototype means:

A new object is created every time the bean is requested.

Example:

```
@Component  
@Scope("prototype")  
public class PaymentService {  
  
    public PaymentService() {  
        System.out.println("Constructor called");  
    }  
}
```

Whenever we request this bean:

```
PaymentService p1 = context.getBean(PaymentService.class);  
PaymentService p2 = context.getBean(PaymentService.class);
```

Spring creates two different objects.

p1 and p2 are different instances.

Prototype Lifecycle Flow

For prototype beans:

```
Container starts
↓
BeanDefinition is read
↓
No object is usually created at startup
↓
Client requests prototype bean
↓
Spring creates a new object
↓
Dependencies are injected
↓
Aware callbacks are called
↓
Initialization callbacks are called
↓
Spring hands object to the client
↓
Spring does not track it for normal destruction
```

Important Difference Between Singleton and Prototype

For singleton:

```
Spring creates it.
Spring stores it.
Spring reuses it.
Spring destroys it.
```

For prototype:

```
Spring creates it.
Spring prepares it.
Spring gives it to the caller.
After that, the caller is responsible for it.
```

Spring manages the beginning of the prototype lifecycle, but not the complete ending.

15. Prototype Destruction Does Automatically

Consider this example:

```
@Component
@Scope("prototype")
public class PaymentService {

    public PaymentService() {
        System.out.println("Constructor called");
    }

    @PostConstruct
    public void init() {
        System.out.println("@PostConstruct called");
    }

    @PreDestroy
    public void destroy() {
        System.out.println("@PreDestroy called");
    }
}
```

Main code:

```
AnnotationConfigApplicationContext context =
    new AnnotationConfigApplicationContext(AppConfig.class);

PaymentService p1 = context.getBean(PaymentService.class);

context.close();
```

Expected output idea:

```
Constructor called
@PostConstruct called
```

Usually, we will not see:

```
@PreDestroy called
```

Why does `@PreDestroy` not run for prototype beans?

Because after Spring creates a prototype bean and gives it to the caller, Spring does not keep tracking that specific object.

For singleton beans, Spring knows:

```
I created this one object.
It is stored in my singleton cache.
When the context closes, I can destroy it.
```

For prototype beans, Spring says:

```
I created this object.
I prepared it.
I gave it to the caller.
Now the caller may use it, store it, discard it, or create many more copies.
I will not track all prototype instances for automatic destruction.
```

Why does Spring not keep a list of every prototype object?

Technically, Spring could keep references to every prototype object.

But that would create a serious problem.

Prototype beans can be created many times.

If Spring stored every prototype object forever, those objects could not be garbage collected.

That could cause memory leaks.

So Spring does not fully manage prototype destruction.

Does `context.close()` destroy prototyp

Usually, no.

`context.close()` destroys singleton beans because Spring has them registered and tracked.

But prototype instances that were already handed out are not automatically destroyed.

So if a prototype bean holds expensive resources, we should clean them manually.

For example:

```
close the resource manually  
use try-with-resources where possible  
avoid putting heavy resource cleanup responsibility inside prototype beans
```

Java can still garbage collect the prototype object when there are no references to it.

But garbage collection and Spring destruction callbacks are not the same thing.

16. Important Case: Singleton Depends on Prototype

Now consider this case:

```
OrderService is singleton.  
PaymentSession is prototype.  
OrderService depends on PaymentSession.
```

Example:

```

@Component
public class OrderService {

    private final PaymentSession paymentSession

    public OrderService(PaymentSession paymentSession) {
        this.paymentSession = paymentSession;
    }
}

```

```

@Component
@Scope("prototype")
public class PaymentSession {

    public PaymentSession() {
        System.out.println("PaymentSession created");
    }

    @PreDestroy
    public void destroy() {
        System.out.println("PaymentSession destroyed");
    }
}

```

Since `OrderService` is singleton, it is created only once.

At the time of creating `OrderService`, Spring creates one `PaymentSession` and injects it.

After that, the same `PaymentSession` instance remains inside `OrderService`.

So even though `PaymentSession` is prototype, it does not automatically mean a new object will be used for every method call inside the singleton.

Question

When the context closes, will Spring call @PreDestroy on PaymentSes

Answer:

No, usually Spring will not automatically call @PreDestroy on that

Reason:

PaymentSession is prototype scoped.
Spring created it, initialized it, and handed it to OrderService.
After that, Spring does not manage its full destruction lifecycle.

Key Learning

Prototype scope means:

new object per request from Spring

It does not mean:

new object every time a singleton method runs

If a singleton needs a fresh prototype instance every time, we need other approaches like:

```
ObjectProvider  
Provider  
lookup method injection  
applicationContext.getBean()
```

Example using `ObjectProvider` :

```

@Component
public class OrderService {

    private final ObjectProvider<PaymentSession> paymentSessionProvider;

    public OrderService(ObjectProvider<PaymentSession> paymentSessionProvider) {
        this.paymentSessionProvider = paymentSessionProvider;
    }

    public void createOrder() {
        PaymentSession session = paymentSessionProvider.getObject();
        System.out.println(session);
    }
}

```

Now a new prototype object can be requested whenever needed.

17. @Lazy and Bean Lifecycle

By default, singleton beans are eagerly created.

That means:

When the Spring container starts, singleton beans are usually created immediately.

But if we mark a singleton bean as `@Lazy`, Spring reads its bean definition but does not create the object immediately.

Example:

```

@Component
@Lazy

```

```
public class PaymentService {

    public PaymentService() {
        System.out.println("PaymentService cre
    }
}
```

Flow:

```
Container starts
    ↓
BeanDefinition is registered
    ↓
Object is not created yet
    ↓
Bean is created only when someone asks for it
```

So for a lazy singleton:

```
BeanDefinition is read at startup.
Actual object creation is delayed.
Dependency injection and initialization also happen later.
```

18. **@PostConstruct** and Circular Dependency

```
@Component
public class A {

    B b;

    public A(B b) {
    }
}
```

```

@Component
public class B {

    A a;

    public B(A a) {
    }

}

```

Here Spring cannot create `A` without `B`, and it cannot create `B` without `A`.
So constructor circular dependency fails.

Where can `@PostConstruct` help?

```

@Component
public class A {
    B b;

    public A(B b) {
        this.b = b;
    }

    @PostConstruct
    public void setB() {
        b.setA(this);
    }
}

@Component
public class B {
    A a;

    public void setA(A a) {

```

```
        this.a = a;
    }
}
```

20. Final Revision Summary

Bean lifecycle is the complete journey of a Spring bean.

The main phases are:

```
BeanDefinition reading
→ object creation
→ dependency injection
→ Aware callbacks
→ BeanPostProcessor before initialization
→ initialization callbacks
→ BeanPostProcessor after initialization
→ bean ready
→ destruction callbacks
```

For singleton beans:

```
Spring manages the full lifecycle from creation to destruction.
```

For prototype beans:

```
Spring creates, injects dependencies, initializes, and gives the object to the caller.
After that, Spring does not automatically destroy it.
```

For `@Lazy` singleton beans:

```
BeanDefinition is read at startup.
Object creation is delayed until the bean is actually needed.
```

For `@PostConstruct`:

It is used to run logic after dependency injection.
It is part of initialization.

21. One-Line Takeaway

Spring Bean Lifecycle is not just object creation. It is the complete process through which Spring discovers, creates, prepares, manages, and destroys a bean.